# 第二讲

- 有限状态系统简介
- 有限状态自动机

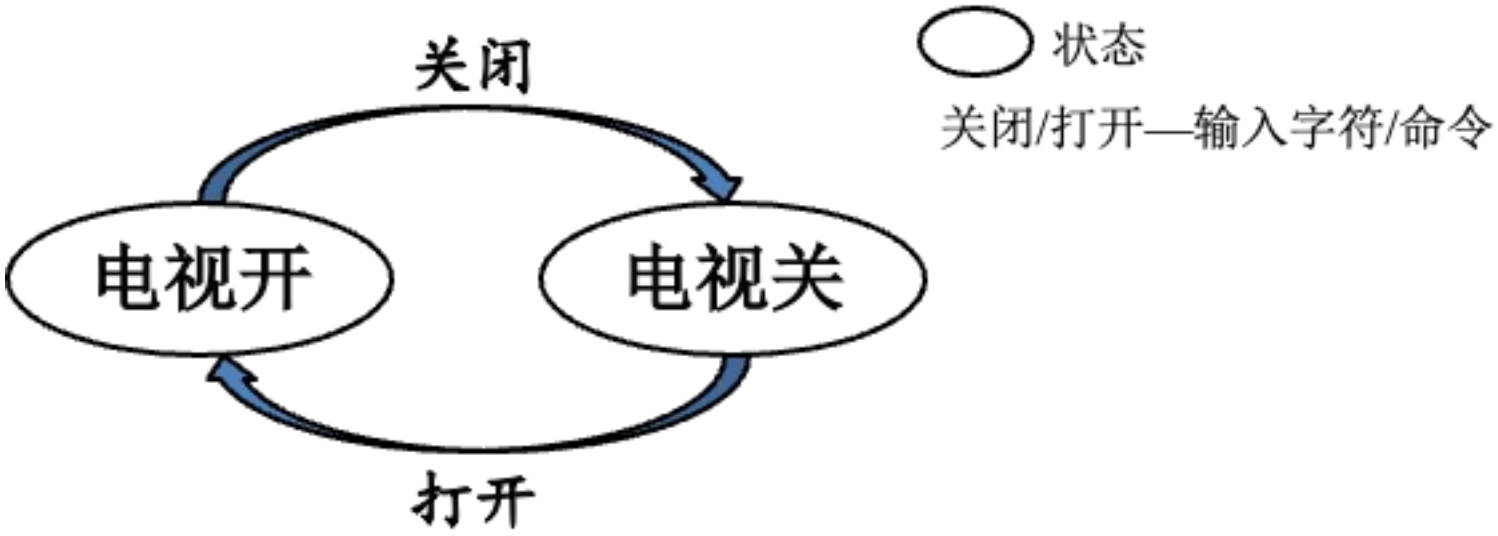
# 有限状态系统

定义：在系统的运行过程中，系统的状态在有限状态间不断变化，每个状态可以迁移到零个或多个状态，系统的输入决定执行哪个状态的迁移。

# 有限状态系统

定义：在系统的运行过程中，系统的状态在有限状态间不断变化，每个状态可以迁移到零个或多个状态，系统的输入决定执行哪个状态的迁移。

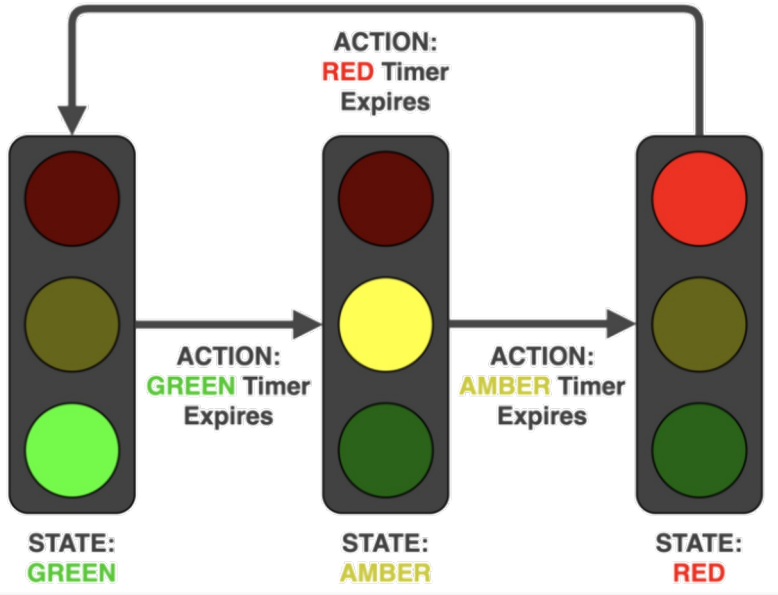
例: 电视遥控器开关



# 有限状态系统

定义：在系统的运行过程中，系统的状态在有限状态间不断变化，每个状态可以迁移到零个或多个状态，系统的输入决定执行哪个状态的迁移。

例: 交通信号灯

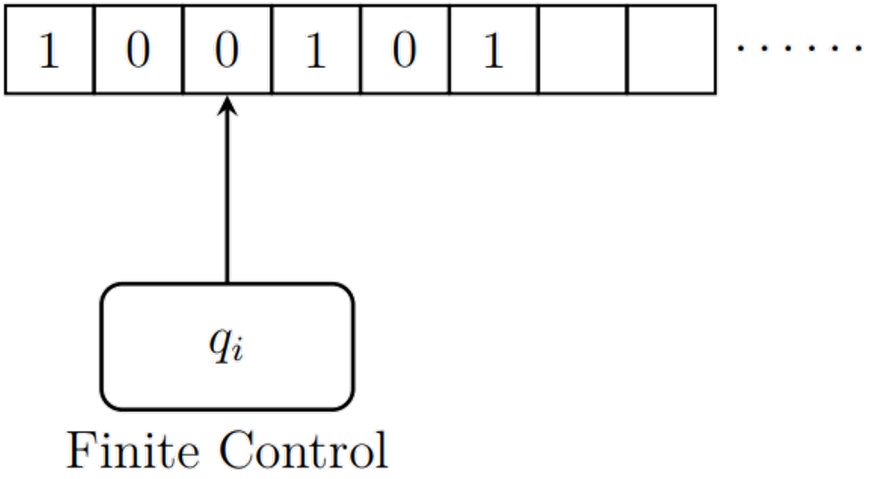


# 有限状态系统

- 数字电路设计
- 电脑游戏设计
- 各种通信协议： **TCP, HTTP, Bluetooth, WiFi**
- 文本搜索
- 词法分析

# 有限自动机抽象装置

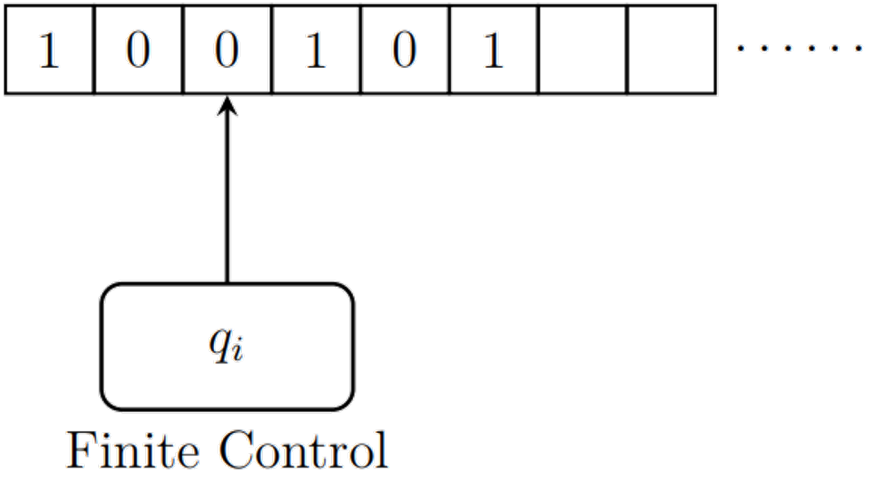
- 一条输入带
- 一个读头
- 一个有穷控制器




- 输入带划分为单元格，每个格子可以放置一个字符，整条输入带可以表示一个符号串。

# 有限自动机抽象装置

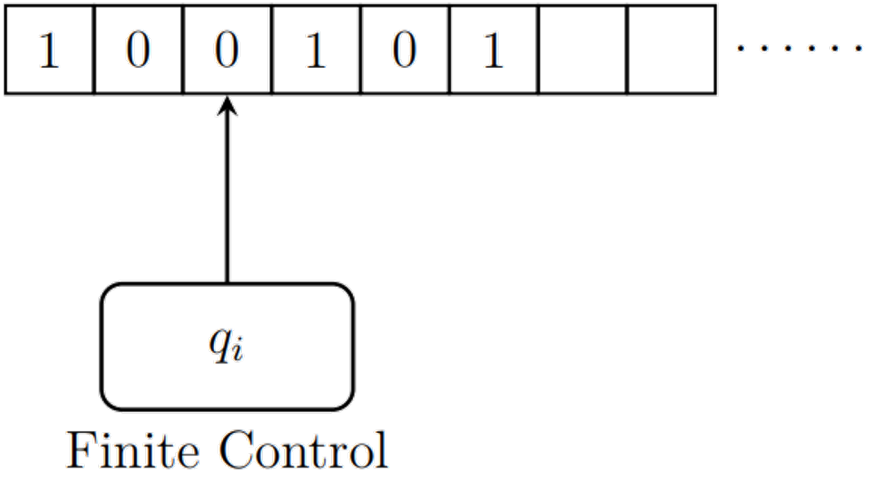
- 一条输入带
- 一个读头
- 一个有穷控制器




- 输入带划分为单元格，每个格子可以放置一个字符，整条输入带可以表示一个符号串。
- 读头可以从左到右扫描输入带，将每次的字符传输给控制器。

# 有限自动机抽象装置

- 一条输入带
- 一个读头
- 一个有穷控制器




- 输入带划分为单元格，每个格子可以放置一个字符，整条输入带可以表示一个符号串。
- 读头可以从左到右扫描输入带，将每次的字符传输给控制器。
- 控制器根据读入的字符和当前的状态，进行状态转移。然后控制读头进行下一个字符的读取。

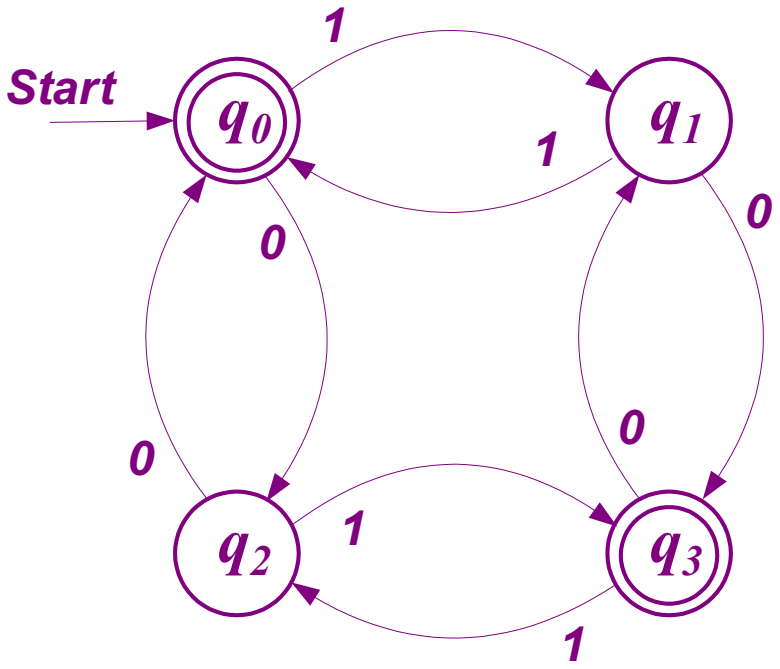
# 第二讲

- 有限状态系统简介
- 有限状态自动机

# 确定有限自动机

✧ 有限自动机的五要素

- 有限状态集
- 有限输入符号集
- 转移函数
- 一个开始状态
- 一个终态集合



# 确定有限自动机

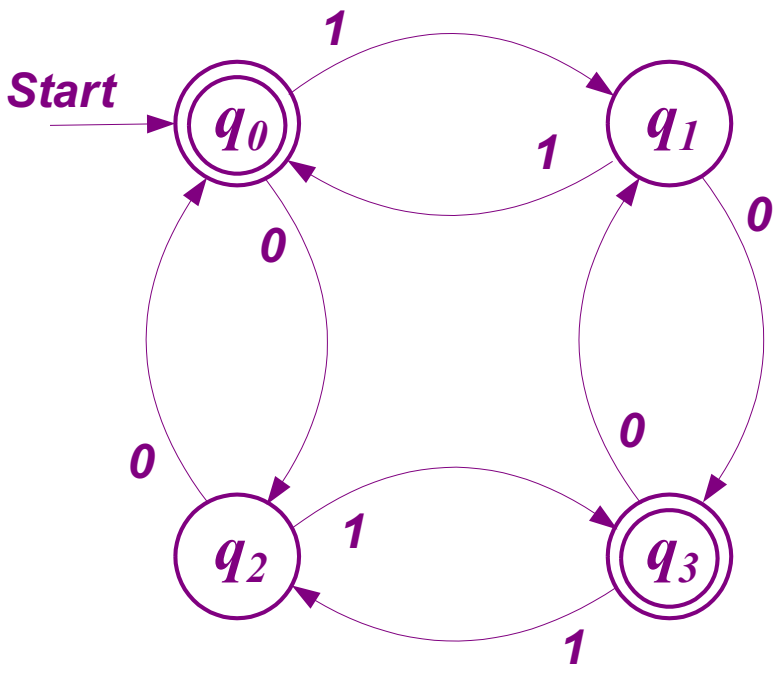
## ✧ 确定有限自动机的形式定义

一个确定有限状态自动机 ***DFA (deterministic finite automata)*** 是一个五元组 $A = (Q, \Sigma, \delta, q_0, F)$.

- 有限状态集
- 有限输入符号集
- 转移函数 $\delta: Q \times \Sigma \to Q$
- 一个开始状态 $q_0 \in Q$
- 一个终态集合 $F \subseteq Q$

# 确定有限自动机

✧ 转移图表示的 DFA




– $Q = \{q_0, q_1, q_2, q_3\}$

– $\Sigma = \{0, 1\}$

– $\delta(q_0,0) = q_2$, $\delta(q_0,1) = q_1$
$\delta(q_1,0) = q_3$, $\delta(q_1,1) = q_0$
$\delta(q_2,0) = q_0$, $\delta(q_2,1) = q_3$
$\delta(q_3,0) = q_1$, $\delta(q_3,1) = q_2$

– $q_0$

– $F = \{q_0, q_3\}$

# 确定有限自动机

✧ 转移表表示的 DFA

| | 0 | 1 |
|---|---|---|
| → *$q_0$ | $q_2$ | $q_1$ |
| $q_1$ | $q_3$ | $q_0$ |
| $q_2$ | $q_0$ | $q_3$ |
| *$q_3$ | $q_1$ | $q_2$ |

– $Q = \{q_0, q_1, q_2, q_3\}$

– $\Sigma = \{0, 1\}$

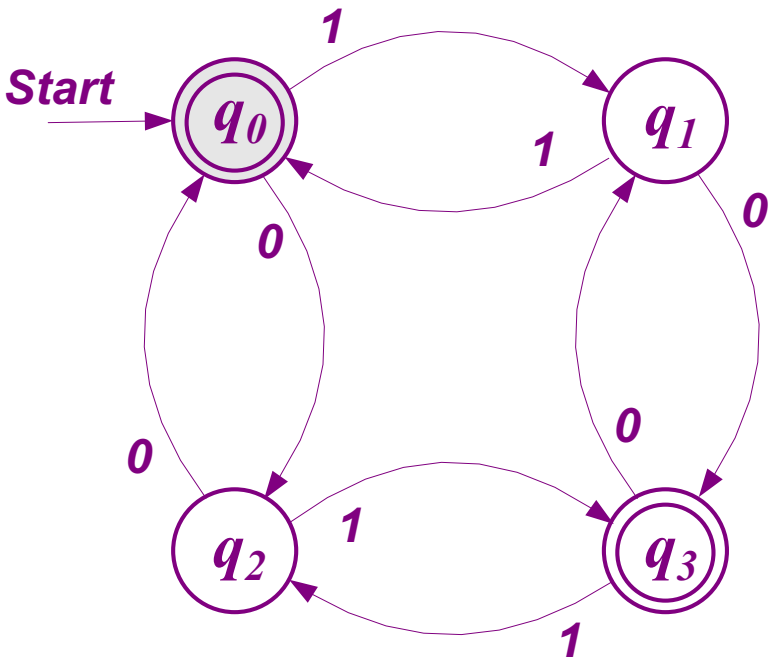
– $\delta(q_0,0) = q_2$, $\delta(q_0,1) = q_1$
$\delta(q_1,0) = q_3$, $\delta(q_1,1) = q_0$
$\delta(q_2,0) = q_0$, $\delta(q_2,1) = q_3$
$\delta(q_3,0) = q_1$, $\delta(q_3,1) = q_2$

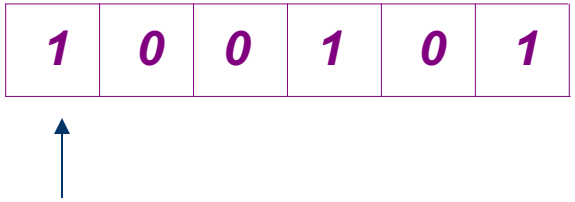
– $q_0$

– $F = \{q_0, q_3\}$

# 确定有限自动机

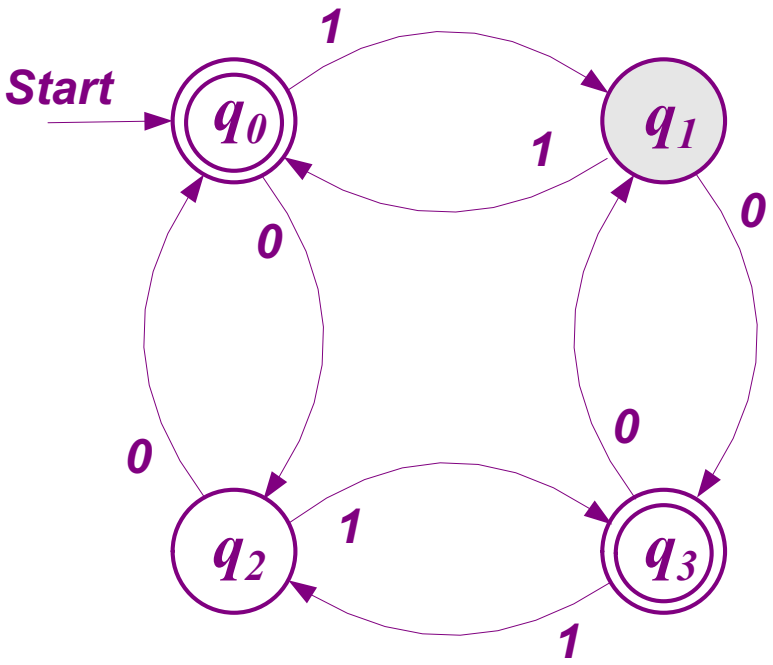
- DFA如何接受输入符号串

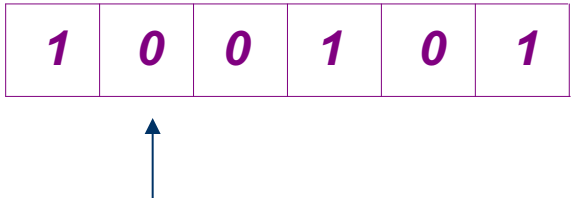


# 确定有限自动机

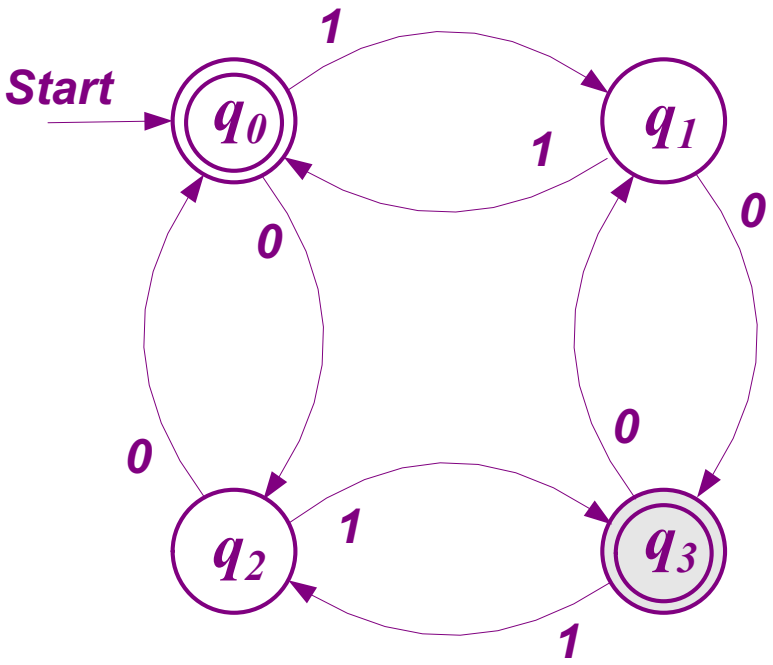
- DFA如何接受输入符号串

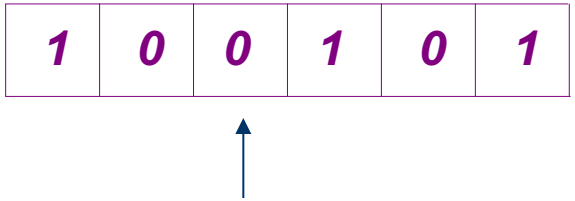


# 确定有限自动机

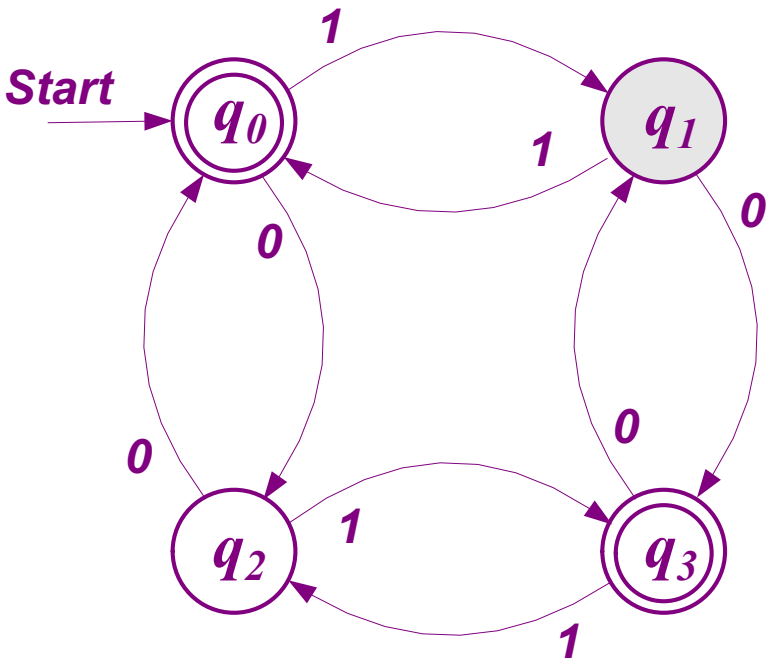
- DFA如何接受输入符号串

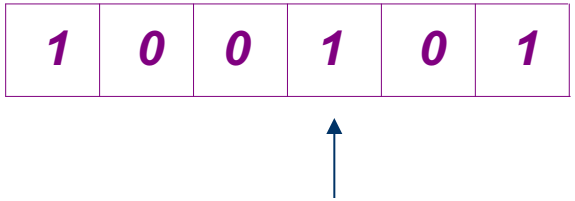


# 确定有限自动机

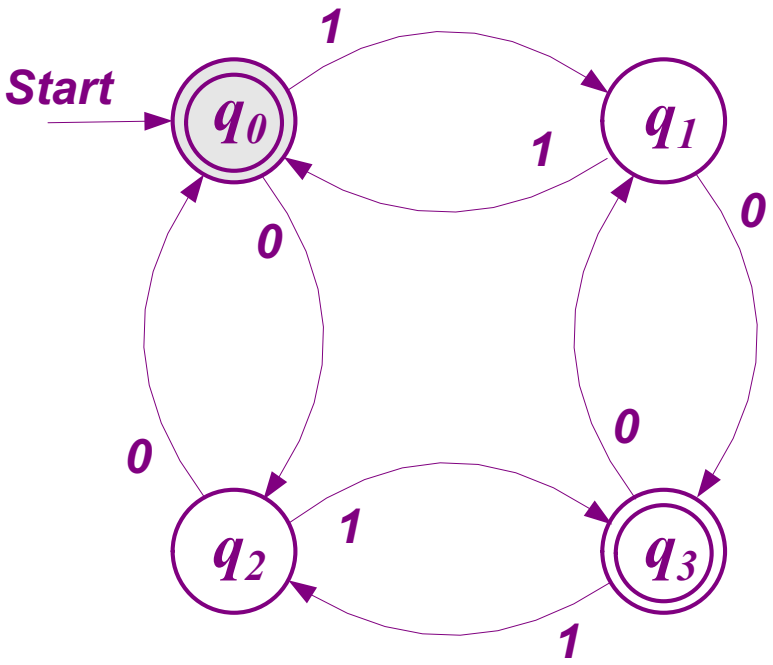
- DFA如何接受输入符号串

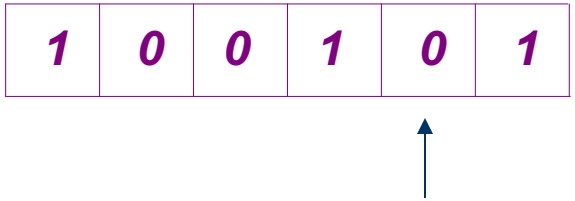


# 确定有限自动机

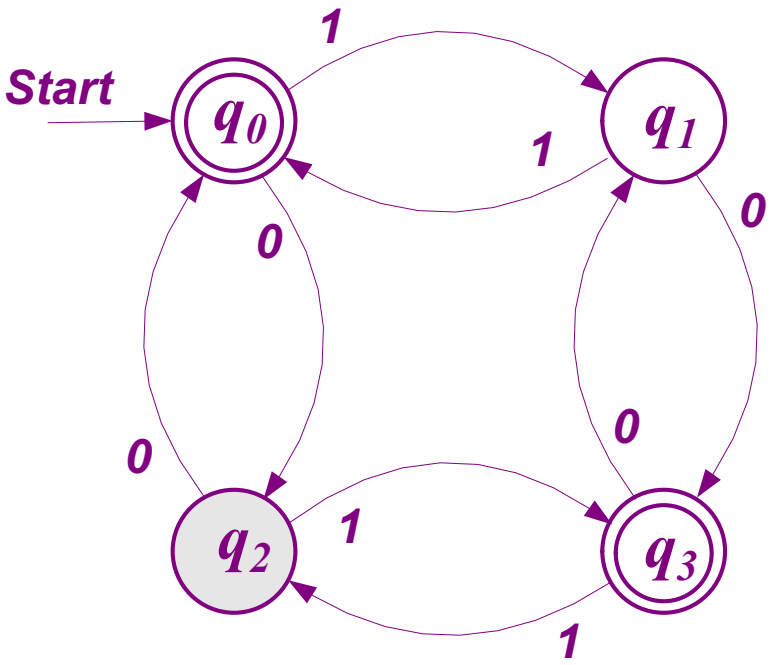
- DFA如何接受输入符号串

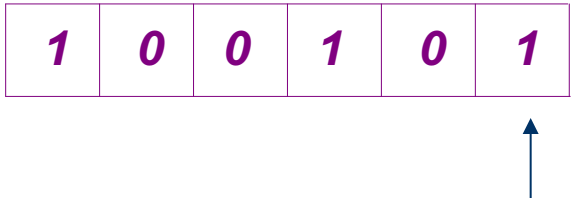


# 确定有限自动机

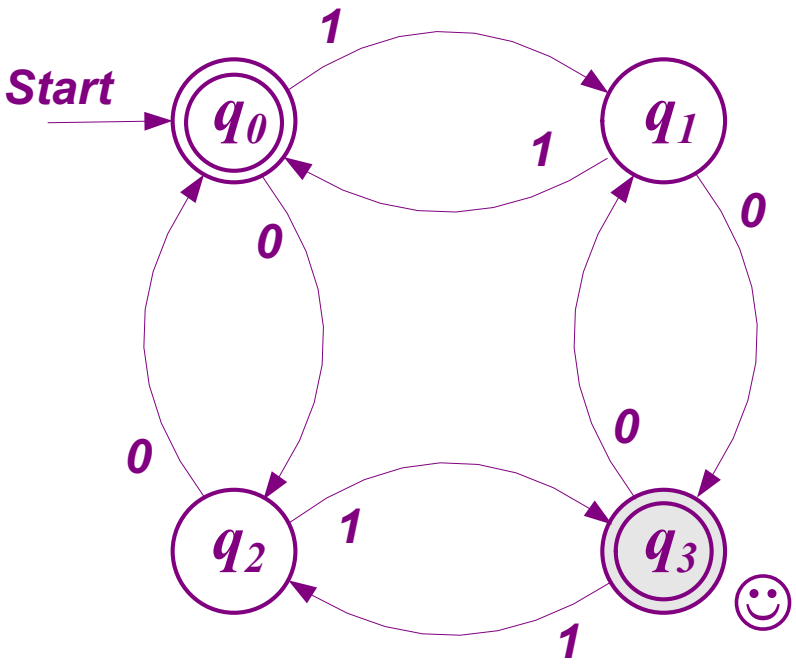
- DFA如何接受输入符号串



# 确定有限自动机

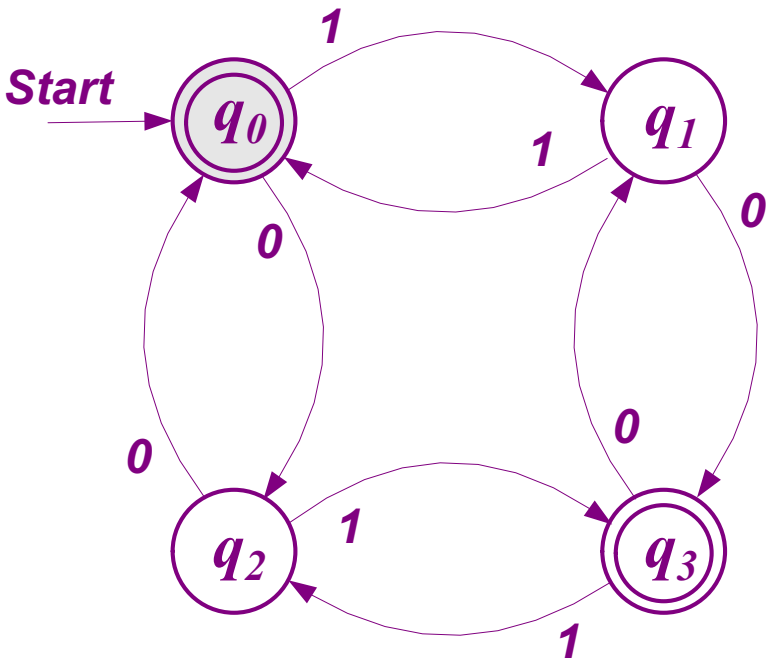
- DFA如何接受输入符号串

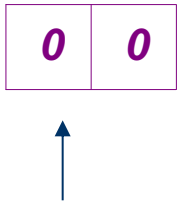



| 1 | 0 | 0 | 1 | 0 | 1 |
|---|---|---|---|---|---|

# 确定有限自动机

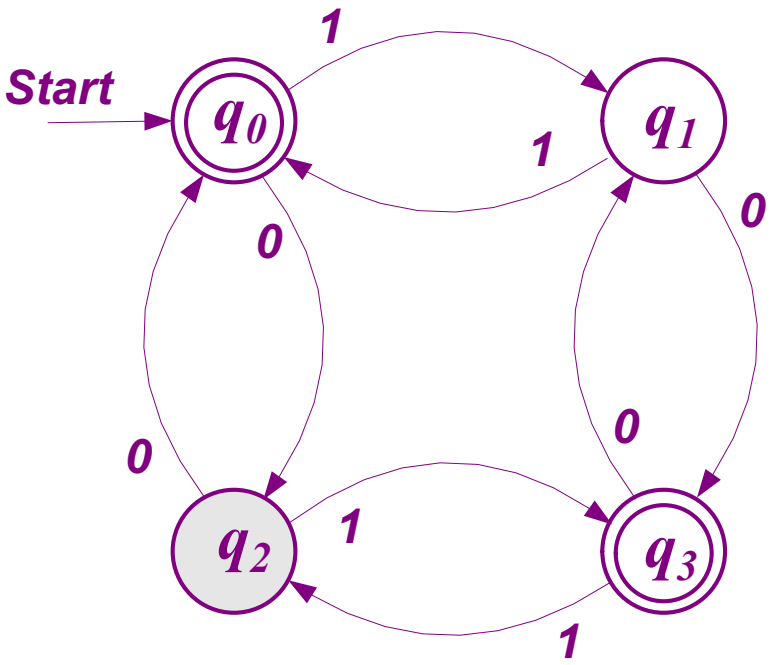
- DFA如何接受输入符号串

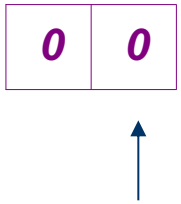


# 确定有限自动机

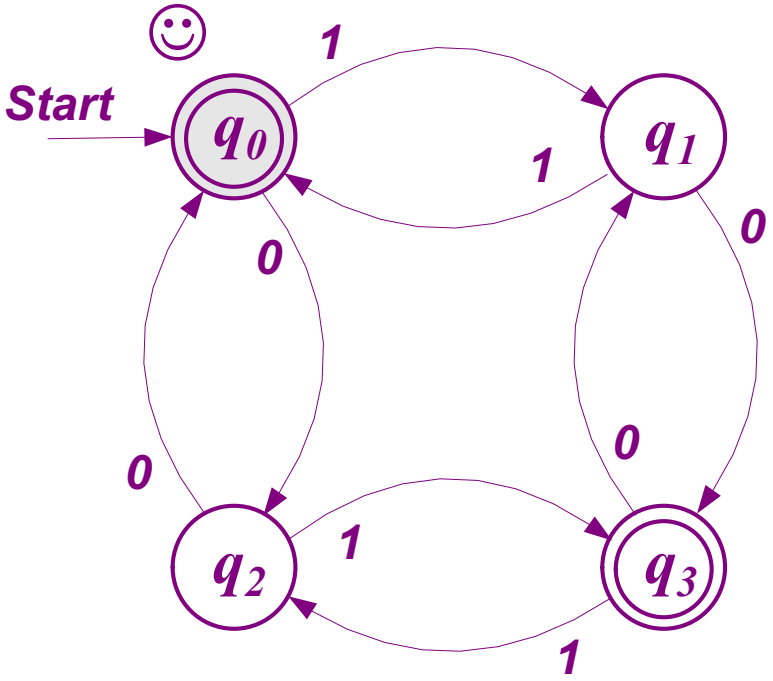
- DFA如何接受输入符号串

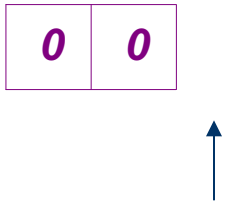


# 确定有限自动机

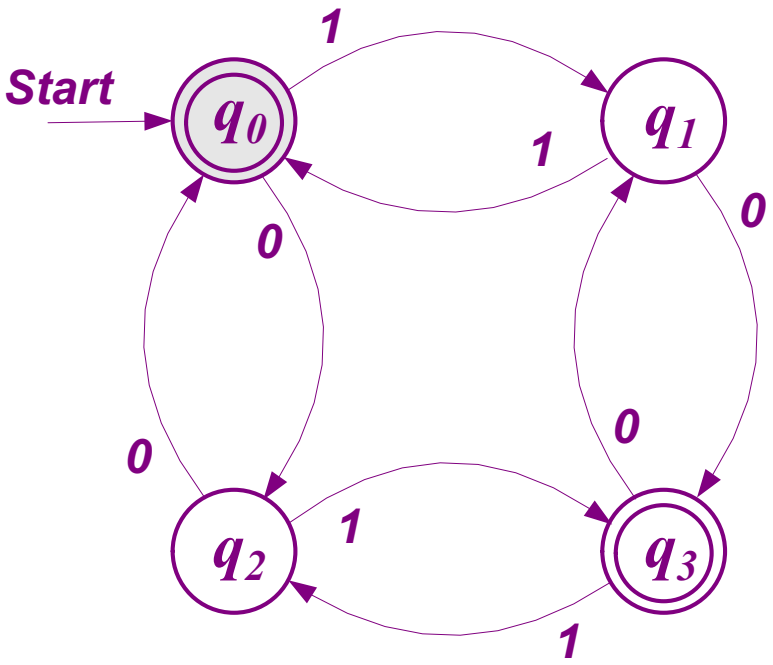
- DFA如何接受输入符号串

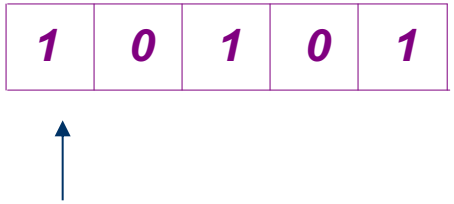


# 确定有限自动机

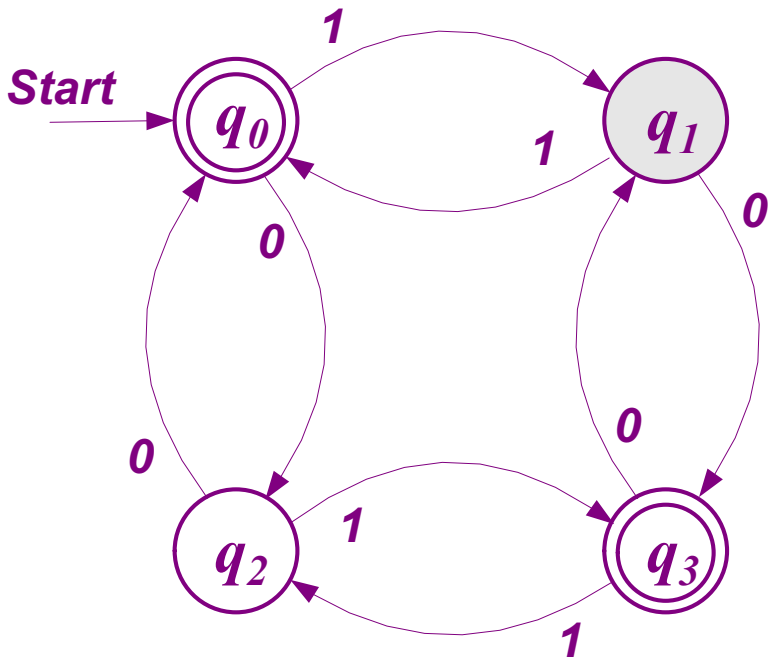
✧ DFA如何接受输入符号串

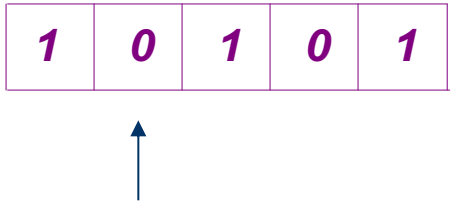


# 确定有限自动机

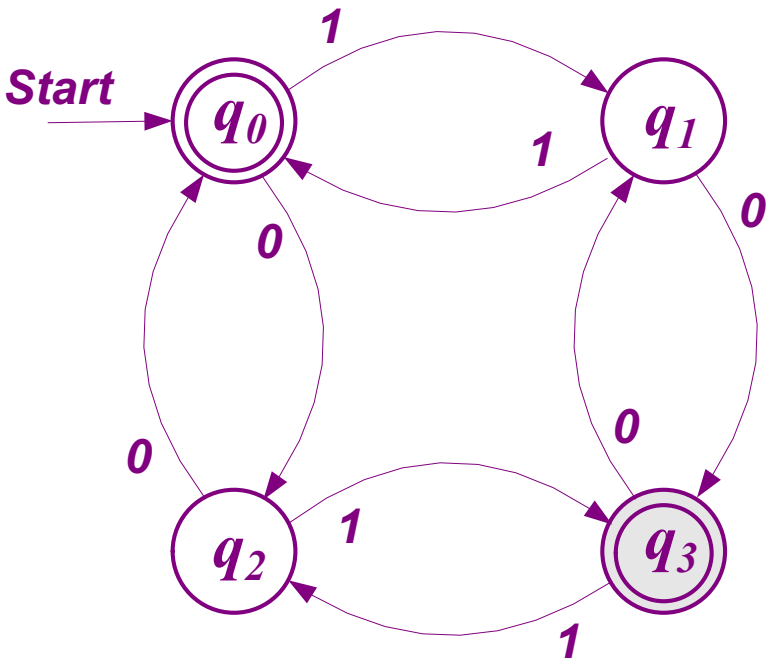
✧ DFA如何接受输入符号串

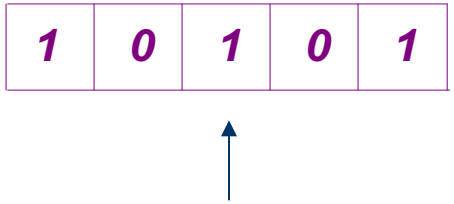


# 确定有限自动机

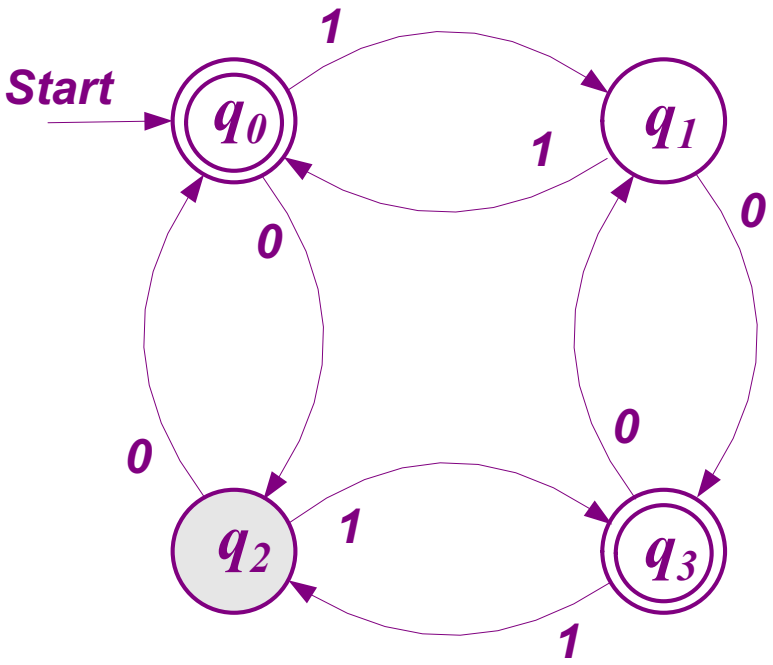
- DFA如何接受输入符号串

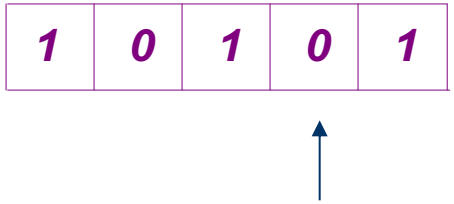


# 确定有限自动机

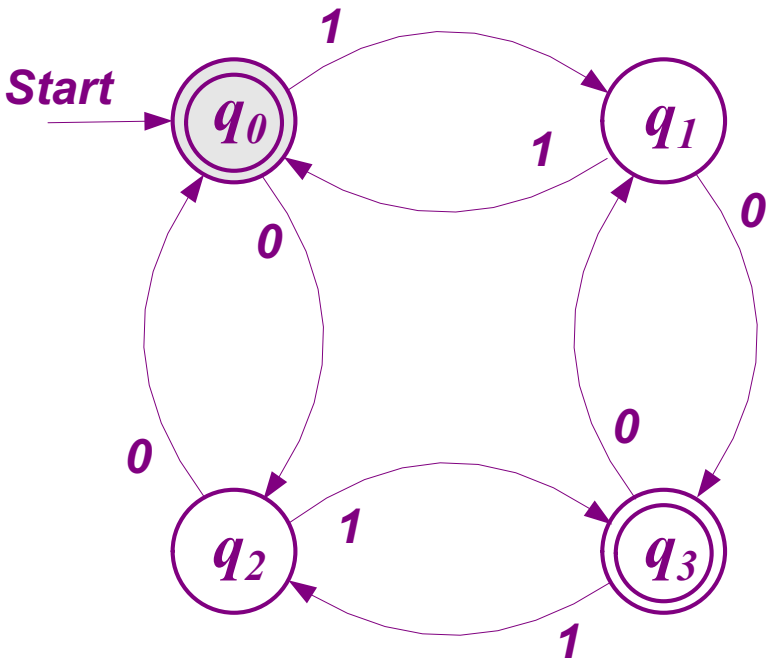
- DFA如何接受输入符号串

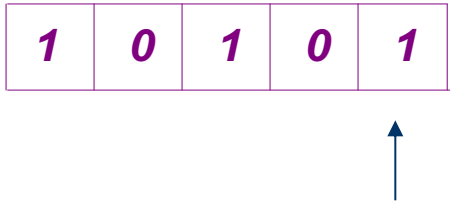


# 确定有限自动机

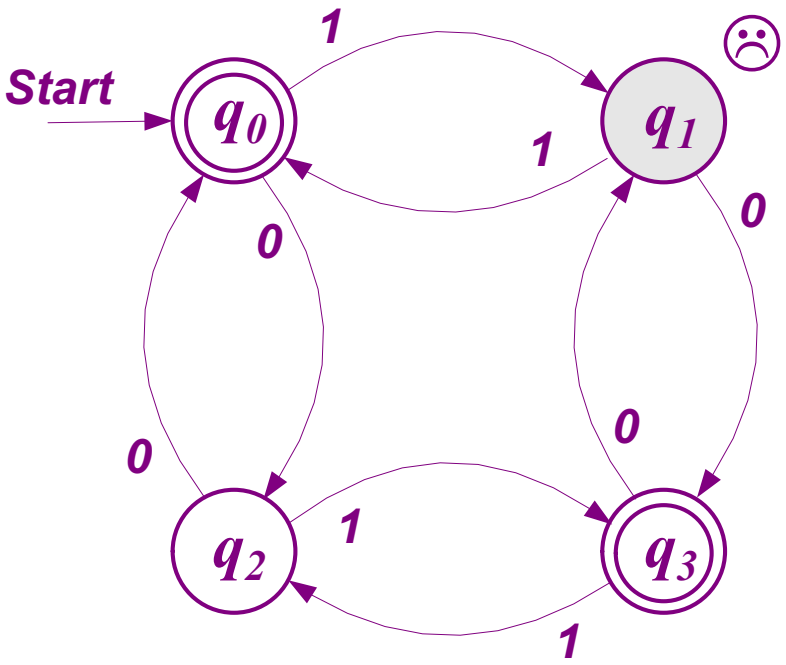
- DFA如何接受输入符号串



# 确定有限自动机

- DFA如何接受输入符号串




| 1 | 0 | 1 | 0 | 1 |
|---|---|---|---|---|

# 确定有限自动机

## ✧ DFA如何接受输入符号串

| | 0 | 1 |
|---|---|---|
| $\rightarrow *q_0$ | $q_1$ | $q_2$ |
| $q_1$ | $q_3$ | $q_0$ |
| $q_2$ | $q_0$ | $q_3$ |
| $q_3$ | $q_3$ | $q_0$ |

给出该DFA在读输入串10010001过程中经历的状态序列。

$$q_0 \xrightarrow{1} q_2 \xrightarrow{0} q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_0$$

$$\xrightarrow{0} q_1 \xrightarrow{0} q_3 \xrightarrow{0} q_3 \xrightarrow{1} q_0$$

# 确定有限自动机

✧ 扩展转移函数适合于输入字符串

– 设一个 DFA $A = (Q, \Sigma, \delta, q_0, F)$

$\delta: Q \times \Sigma \to Q$

– 扩充定义 $\delta'$: $Q \times \Sigma^* \to Q$

对任何$q \in Q$，定义:

1 $\delta'(q, \varepsilon) = q$

2 若$w = xa$, 其中 $x \in \Sigma^*$, $a \in \Sigma$, 则

$$\delta'(q, w) = \delta(\delta'(q, x), a)$$

# 确定有限自动机

✧ 扩展转移函数适合于输入字符串

1 $\delta'(q, \varepsilon) = q$

2 若$w = xa$, 其中 $x \in \Sigma^*$, $a \in \Sigma$, 则

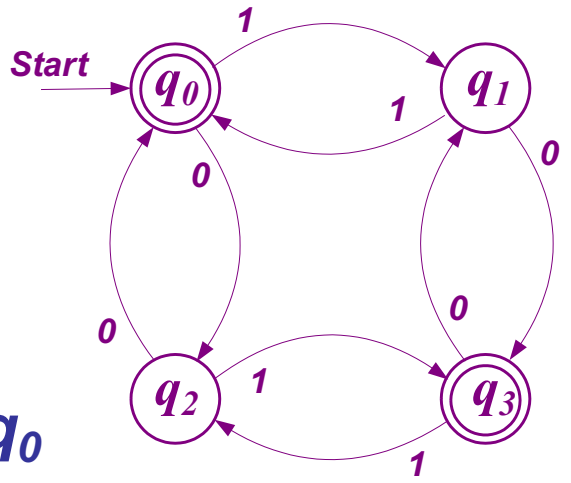
$$\delta'(q, w) = \delta(\delta'(q, x), a)$$

$$\begin{aligned}\delta'(q, w) &= \delta(\delta'(q, a_0 a_1 \cdots a_{n-1}), a_n) \\ &= \delta(\delta(\delta'(q, a_0 a_1 \cdots a_{n-2}), a_{n-1}), a_n) \\ &= \ldots \\ &= \delta(\delta(\cdots \delta(\delta'(q, \varepsilon), a_0) \cdots), a_{n-1}), a_n)\end{aligned}$$

# 确定有限自动机

✧ 扩展转移函数适合于输入字符串

| | 0 | 1 |
|---|---|---|
| → *$q_0$ | $q_2$ | $q_1$ |
| $q_1$ | $q_3$ | $q_0$ |
| $q_2$ | $q_0$ | $q_3$ |
| *$q_3$ | $q_1$ | $q_2$ |




– 举例

$\delta'(q_0, \varepsilon) = q_0$

$\delta'(q_0, 0) = \delta(q_0, 0) = q_2$

$\delta'(q_0, 00) = \delta(q_2, 0) = q_0$

$\delta'(q_0, 001) = \delta(q_0, 1) = q_1$

$\delta'(q_0, 0010) = \delta(q_1, 0) = q_3$

# DFA的语言与正则语言

若$D = (Q, \Sigma, \delta, q_0, F)$是一个**DFA,**则D接受的语言为

$$L(D) = \{w \in \Sigma^* | \hat{\delta}(q_0, w) \in F\}$$

即所有能使**DFA**从开始状态到达接受状态的符号串集合。$\delta$^与$\hat{\delta}$都可以表示扩展转移函数。

定义: 如果语言**L**是某个DFA **D**的语言，则称其为正则语言。

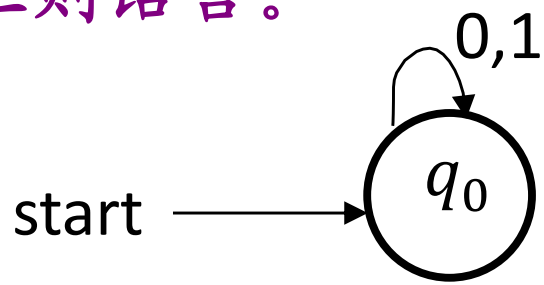
# DFA的语言与正则语言

若$D = (Q, \Sigma, \delta, q_0, F)$是一个**DFA**,则D接受的语言为

$$L(D) = \{w \in \Sigma^* | \hat{\delta}(q_0, w) \in F\}$$

即所有能使**DFA**从开始状态到达接受状态的符号串集合。$\delta$^与$\hat{\delta}$都可以表示扩展转移函数。

定义: 如果语言**L**是某个DFA **D**的语言，则称其为正则语言。




∅的DFA: 无接受状态，即该DFA不接受任何符号串（包括空串）。

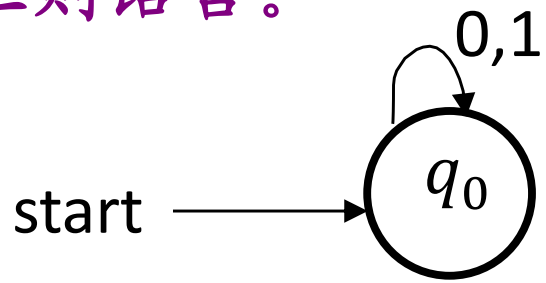
# DFA的语言与正则语言

若$D = (Q, \Sigma, \delta, q_0, F)$是一个**DFA**,则D接受的语言为

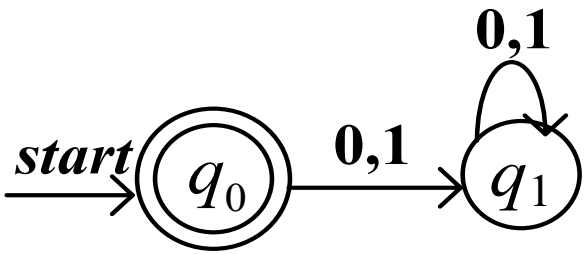
$$L(D) = \{w \in \Sigma^* | \hat{\delta}(q_0, w) \in F\}$$

即所有能使**DFA**从开始状态到达接受状态的符号串集合。$\delta$^与$\hat{\delta}$都可以表示扩展转移函数。

定义: 如果语言**L**是某个DFA **D**的语言，则称其为正则语言。




∅的DFA: 无接受状态，即该DFA不接受任何符号串（包括空串）。




{ε}的DFA。

单选题 1分

设置

两个不同的有穷自动机可能识别同一个语言。

A 正确

B 错误

提交

单选题 1分

设置

由字符0和1构成且含有奇数个1的DFA，至少需要几个状态?

A 2

B 3

C 1

D 4

提交

单选题 2分

设置

由字符0和1构成且含有奇数个1和偶数个0的DFA，至少需要几个状态?

A 2

B 3

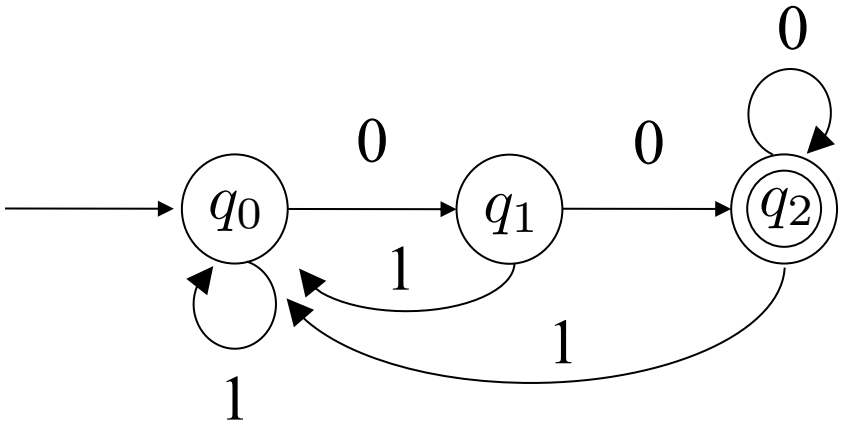
C 1

D 4

提交

# 确定有限自动机

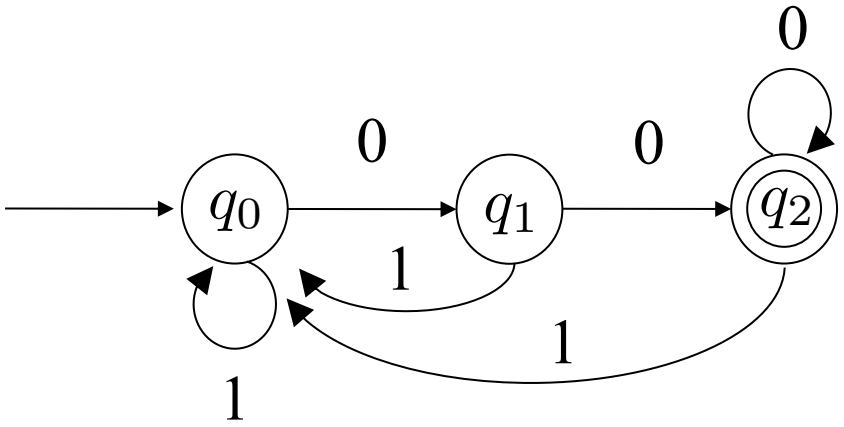
例1：设计定义在字母表{0, 1}上的所有以00结尾的串的集合的DFA。




解：首先画出三个状态，接受两个连续的0到达终止状态。由于DFA要求每个节点的出边必须包含字母表上的所有符号，因此，必须要补上所缺的标为1出边，同时也不能忘记终止状态还有一个标为0的出边。最后补上状态编号。

# 确定有限自动机

例1：设计定义在字母表{0, 1}上的所有以00结尾的串的集合的DFA。

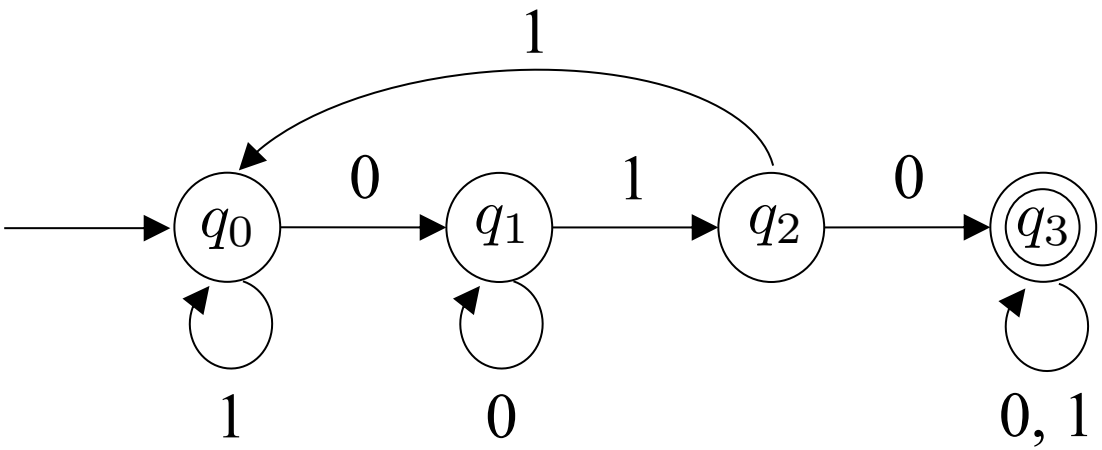



解：三个状态其实都有明确的含义。$q_0$ 表示出现第一对连续的0之前的任意字符串。$q_1$ 表示第一对连续的0之中的第一个0，一旦在该状态下碰到了1，必须转回到 $q_0$。$q_2$ 则是终止状态，可以接受一对连续的0，也可以接受更多的连续的0。

# 确定有限自动机

例2：设计DFA接受以下语言

$$L = \{x | x \in \{0,1\}^* \wedge x\text{中至少包含一个子串}010\}$$

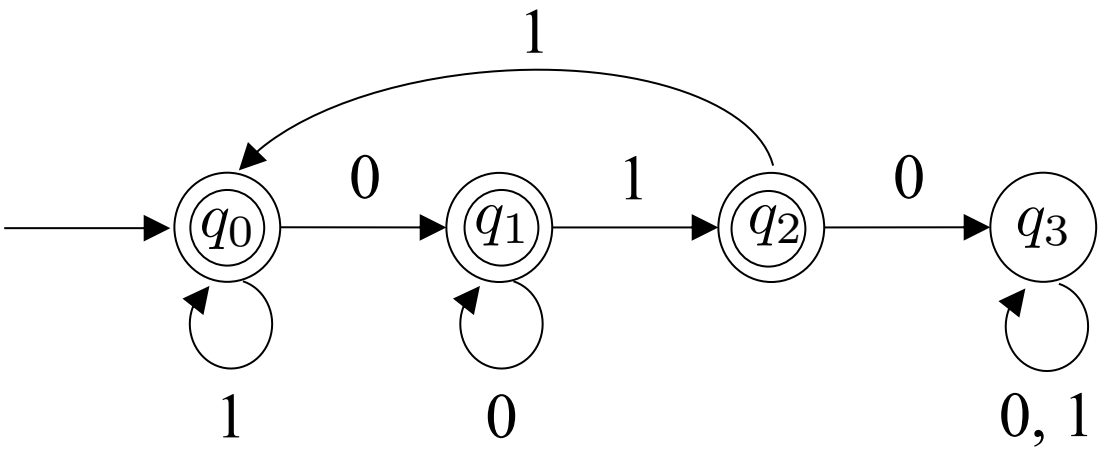



解：这道题可以采用类似与例1的技巧，首先确定主线上的节点和边，然后再补所缺的出边。一定要想清楚出边应该指向哪个节点。做完后可以列举几个符合条件的串检查一下。

# 确定有限自动机

例3：设计DFA接受以下语言

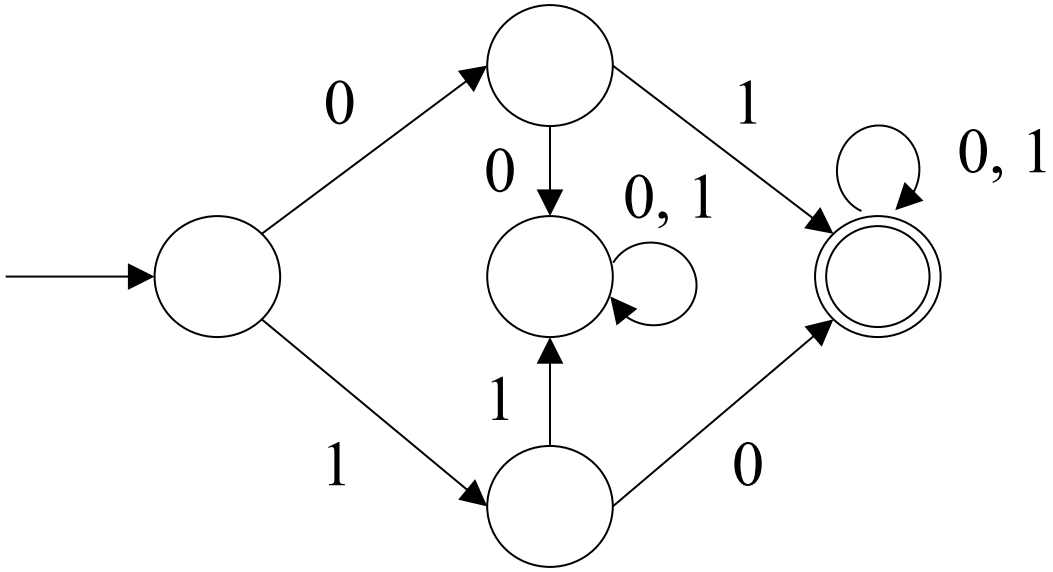
$$L = \{x | x \in \{0,1\}^* \wedge x\text{中不包含子串}010\}$$




解：这道题看上去似乎无从下手，怎么表示“不包含”？其实可以从补集的角度去考虑，体现在DFA上，就是保留例2中DFA的结构，只不过互换终结状态和非终结状态。

# 确定有限自动机

例4：设计DFA表示所有长度至少为2且头两个字符不同的0和1组成的串的集合。

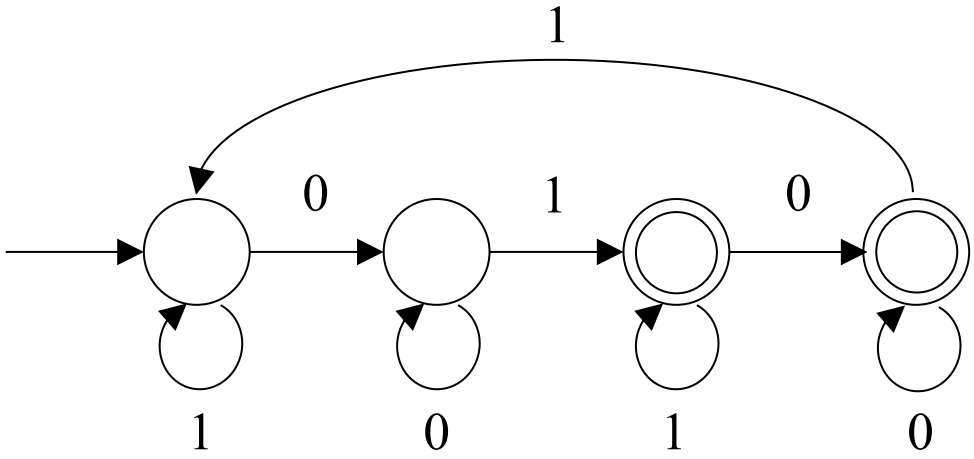



解：思路和前面一样，首先完成主线的构建，然后再对每个节点补出边。这里专门引入了一个节点充当“垃圾箱”，收无效的字符串，这是因为DFA要求每个节点必须有出边。

# 确定有限自动机

例5：设计DFA表示以下语言

$$L = \{x | x \in \{0,1\}^* \wedge \text{子串}01\text{在}x\text{中出现的次数为奇数}\}$$

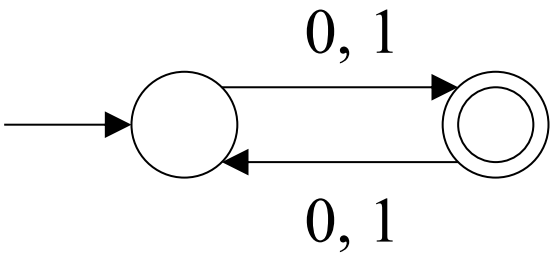



解：此题的难点在于如何实现奇数。基本的策略是即将形成偶数个01子串时就远离终结状态。需要注意的是，该串并不子串01的幂，只是包含子串01，所以还有其他的子串。初始节点的含义是包含了偶数个子串01。

# 确定有限自动机

例6：设计DFA表示以下语言

$$L = \{x | x \in \{0,1\}^* \wedge 0\text{的个数与}1\text{的个数之和为奇数}\}$$

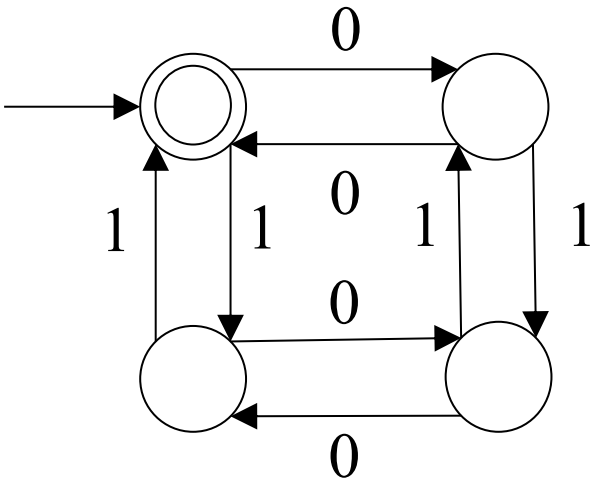



解：这道题比较简单。题目等价于字符串的长度是奇数，这样就类似于电灯开关，比如按奇数次才处于开的状态。因此，很容易设计相应的DFA。

# 确定有限自动机

例7：设计DFA表示以下语言

$L = \{x | x \in \{0,1\}^* \wedge 0$的个数是偶数，$x$的长度也是偶数$\}$



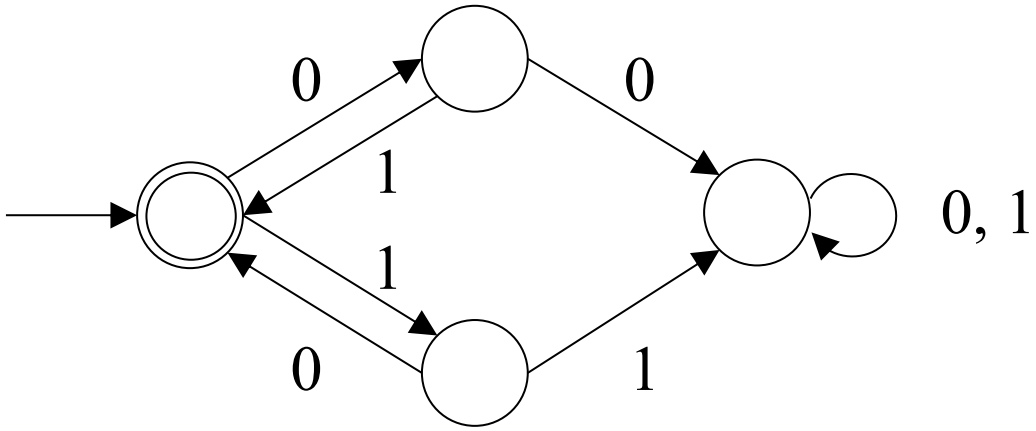

解：题目等价于0的个数是偶数，1的个数也是偶数。根据0和1的奇、偶性，可以划分为4种状态，

相互关联。这样就比较容易设计DFA了。

# 确定有限自动机

例8：设计DFA表示以下语言

$L = \{x | x \in \{0,1\}^* \wedge x$中包含相同个数的0和1$\wedge$
$x$的每个前缀0和1的个数之差不超过1$\}$

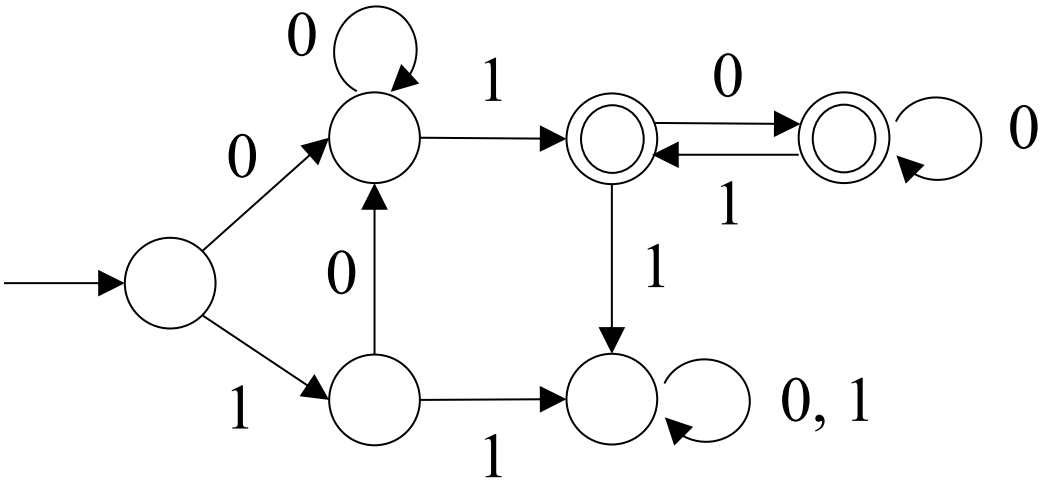



解：这道题的难点是如何实现每个前缀中0和1的个数之差不超过1。实现起来，就是只要出现0，紧接着要出现1，反之亦然。再加上垃圾回收节点，整个DFA就容易构建了。

# 确定有限自动机

例9：设计DFA表示以下语言

$L = \{x | x \in \{0,1\}^* \wedge x$包含子串01但不包含子串11$\}$




解：这道题看上去很复杂，既有包含关系，又有不包含关系。但基本策略不变，还是先把主线上的节点和连线确定，再补齐所有的出边。

# 确定有穷自动机(回顾)

例10：设计DFA表示所有被3整除的二进制数。

解：既然是二进制数，那么实际上就是字母表是{0, 1}的字符串集合。DFA在处理字符串时，往往采用 $w = xa$ 的形式，即在当前状态下（接受了 $x$）再接受一个符号进入新的状态。

**我们可以把 $x$ 看成是一个二进制数，如果再追加一个0，相当于乘以2**。例如，二进制数“11”表示十进制数“3”，追加一个0之后，二进制数“110”表示十进制数“6”。同理，**在二进制数上追加一个1，相当于乘以2再加上1**。例如，二进制数“111”表示十进制数7。

通过上述策略，我们便建立了二进制数和DFA之间的直接关联。接下来看怎么处理“被3整除”。

# 确定有穷自动机(回顾)

例10：设计DFA表示所有被3整除的二进制数。

解：根据整除关系，我们可以把所有的整数分成三类：被3除余数为0、余数为1和余数为2。这三类可以分别对应DFA中的三个状态节点。

接下来看状态节点之间的转移情况。

| X%3 | X0%3 | X1%3 |
|---|---|---|
| 0 (3K) | 0 (6K) | 1 (6K+1) |
| 1 (3K+1) | 2 (6k+2) | 0 (6K+3) |
| 2 (3K+2) | 1 (6K+4) | 2 (6K+5) |

因此，可以根据上表来设计DFA。

